TP 2 : Manipulation de tableaux de données avec `Numpy`

Ce TP propose d'utiliser le langage de programmation Python pour simuler des courbes et traiter des fichiers de données. Il inclut l'utilisation des modules `matplotlib` et `numpy`.

Partie 1 : Simulation de courbe et recherche de points caractéristique

Exercice 1 : Titrage de l'acide formique par une solution d'hydroxyde de sodium

On souhaite modéliser le titrage de 20 mL d'une solution d'acide formique (C_A = $5{,}00 \cdot 10^{-2}$ mol.L^{-1}) par l'ajout d'une solution d'hydroxyde de sodium (C_B = $8{,}00 \cdot 10^{-2}$ mol.L^{-1}) selon la réaction :

$$HCOOH_{(aq)} + OH^-_{(aq)} + \rightleftharpoons HCOO^-_{(aq)} + H_2O_{(l)}$$

Données : Constante d'acidité de l'acide formique : $K_A = 10^{-3,75}$

Constante de la réaction d'autoprotolyse de l'eau : $K_E = 10^{-14}$

A/ Représentation graphique de la courbe de titrage et de sa dérivée

Utilisation de `numpy` pour créer un N-darray

Plutôt que d'utiliser la commande `for`, il est souvent plus simple d'exploiter les commandes du module `numpy` pour créer des tableaux de données. L'utilisation de `numpy` (souvent abrégé en `np`) permet de générer des tableaux de N-dimensions appelé N-darray. Contrairement aux listes Python classiques, ces tableaux offrent une manipulation plus rapide et plus efficace. Cependant, leur manipulation diffère légèrement de celle des listes classiques. Par exemple, dans une liste ou un tableau à deux dimensions, extraire le 4e élément de la 3e colonne se code en Python comme suit :

```
x[3][2]
```

Avec numpy, cette opération s'écrit :

```
x[3, 2]
```

Les tableaux peuvent être générés facilement grâce aux commandes suivantes :

Créer un tableau avec des valeurs réparties uniformément :

Utilisez `np.linspace` pour générer un tableau avec un nombre défini de points répartis entre deux valeurs :

```
np.linspace(début, fin_inclu, nombre_de_points)
```

Exemple :

```
np.linspace(0, 10, 5) # renvoie : array([0., 2.5, 5., 7.5, 10.])
```

Créer un tableau avec des pas réguliers :

Utilisez `np.arange` pour générer un tableau avec un pas fixe entre deux valeurs :

```
np.arange(début, fin exclu, pas)
```

Exemple :

```
np.arange(0, 10, 2) # renvoie : array([0, 2, 4, 6, 8])
```

Utilisation de `numpy` pour calculer une dérivée

Pour calculer la dérivée d'une courbe, vous pouvez utiliser la commande `np.gradient()` fournie par le module **`numpy`**. Cette commande calcule une approximation numérique de la dérivée en évaluant les différences entre les points consécutifs du tableau :

```
np.gradient(variable)  # variable : le tableau ou ndarray pour lequel la dérivée doit être calculée.
```

Exemple :

```
import numpy as np
x = np.linspace(0, 10, 100) # génère 100 points entre 0 et 10
y = x**2                    # fonction y = x²
dy_dx = np.gradient(y, x)   # calcul de la dérivée dy/dx
```

Représentation d'un nuage de points avec Matplotlib

Pour afficher des données expérimentales ou simulées, un nuage de points est souvent plus adapté qu'un tracé continu. Vous pouvez utiliser la commande plt.scatter() du module Matplotlib pour cela :

```
plt.scatter(x ,y,  paramètres_graphiques)  # paramètres graphiques : color (ou c) définit la couleur du tracé ; marker spécifie le type de marqueur. Voici quelques options : plus '+', rond 'o', croix 'x', carré 's', triangle 'v', étoile '*', point '.' ; markersize : taille des marqueurs.
```

Exemple :

```
import matplotlib.pyplot as plt
x = np.linspace(0, 10, 20) # génère 20 points
y = x**2                   # fonction y = x²
plt.scatter(x, y, color='blue', marker='o', s=50)
plt.title('Nuage de points')# ajout d'un titre
plt.xlim([0,10])           # limite de l'axe x
plt.ylim([0,100])          # limite de l'axe y
plt.xlabel('x')            # nom de l'axe x
plt.ylabel('y')            # nom de l'axe y
plt.show()
```

Notes :

……

……

Instructions :

1. Modélisation des points expérimentaux théoriques de la courbe de dosage pH = f(V_B).

☞ Pour cela :

- *Définir un tableau de volumes ajoutés compris entre 0,1 et 20 mL avec un pas de 0,1 mL.*
- *Définir les concentrations introduites au cours du dosage :*

$$C_{A,t} = C_{A,0} * V_A / (V_A + V_B)$$

$$C_{B,t} = C_{B,0} * V_B / (V_A + V_B)$$

- *Calculez le taux d'avancement volumique à partir de la résolution de l'équation du second degré :*

$$\xi^2 - (C_{A,t} + C_{B,t} + K_E/K_A)\xi + C_{A,t}.C_{B,t} = 0$$

La valeur de ξ sera alors calculé comme suit :

$$\xi = \frac{-b - \sqrt{\Delta}}{2a}$$

- *En déduire la valeur du pH*

2. Calculer la dérivée première de la courbe pH = f(V_B)
3. Représentez la courbe modélisé et sa dérivée première.

☞ Pour cela :

- *Indiquer les points expérimentaux sur le graphique en utilisant des marqueurs triangles*
- *Fixer les limites des axes : axe des abscisses (x) : [0,20] mL et axe des ordonnées (y) : [0,14] unités de pH.*
- *Ajouter le nom des axes, le titre du graphique et la légende.*

Notes :

……
……
……
……
……
……
……
……
……
……
……
……
……

B/ Recherche et affichage graphique du volume à l'équivalence

Le volume à l'équivalence peut être déterminé grâce à la méthode des dérivées : le maximum de la dérivée de la courbe correspond aux coordonnées du volume à l'équivalence.

Recherche d'extremum dans un N-darray.

L'un des avantages du module NumPy est sa capacité à manipuler facilement des tableaux de données (*ndarray*). Avec les commandes suivantes, vous pouvez localiser les extremums et leurs positions :

```
np.max(ndarray) ou np.min(ndarray)  # renvoie la valeur maximale ou minimale d'un tableau.
np.argmax(ndarray) ou np.argmin(ndarray)  # renvoie l'indice de la position de la valeur maximale ou minimale.
```

Une fois le volume à l'équivalence identifié, vous pouvez afficher les points correspondants sur le graphique :

```
plt.vlines(x, ymin, ymax, paramètres)  # pour tracer une ligne verticale.
plt.hlines(y, xmin, xmax, paramètres)  # pour tracer une ligne horizontale.
```

Instructions :

1. Déterminer et afficher dans la console les coordonnées du point à l'équivalence

☞ Pour cela :

- *Déterminer l'indice de la position du maximum de la dérivée puis chercher les valeurs de V_B et pH pour cette position.*

2. Faire apparaitre sur la courbe les coordonnées du volume à l'équivalence.

Exercice 2 : Etude la loi d'Ostwald

Le taux d'avancement de la dissociation d'un acide faible dans l'eau dépend de sa concentration : plus l'acide est dilué, plus le taux de dissociation est important. Nous allons démontrer cette loi, appelée loi d'Ostwald, dans le cas d'une solution d'acide formique avec une concentration comprise entre 10^{-1} et 10^{-8} mol.L^{-1}.

Donnée : *Constante d'acidité :* $K_A = 10^{-3,75}$

Instructions :

1. Tracer la courbe du taux d'avancement en fonction de la concentration

☞ Pour cela :

- *Générer un tableau de 10 000 valeurs de concentration initiale allant de 10^{-1} à 10^{-8} mol/L.*
- *Calculer le taux d'avancement à partir de l'avancement volumique, obtenu par la résolution de l'équation du second degré.*
- *Représenter graphiquement le taux d'avancement en fonction de la concentration sur une échelle logarithmique pour l'axe des abscisses en utilisant la commande*

```
plt.xscale('log')
```

- Ajoutez un titre et des noms d'axes au graphique.

Nous allons maintenant déterminer les concentrations limites :

- **Au-dessus** de laquelle la réaction est peu avancée (avancement inférieur à 10 %).
- **Au-dessous** de laquelle la réaction est quantitative (avancement supérieur à 90 %).

Les valeurs 10 % et 90 % ne correspondant pas nécessairement à des valeurs exactes du tableau, il sera nécessaire de définir une variable calculant la différence entre les taux d'avancement du tableau et ces valeurs d'intérêt.

Instructions :

1. Déterminer et afficher les concentrations limites

☞ Pour cela :

- *Calculer la différence entre les taux d'avancement du tableau et les valeurs cibles (10 % et 90 %).*
- *Trouver les indices des minimums de ces différences afin d'identifier les concentrations limites.*
- *Afficher dans la console les valeurs correspondantes de concentration et de taux d'avancement.*

2. Représenter graphiquement ces points caractéristiques